

MotionLink : Système d'Interaction par Gestes Manuels en Temps Réel pour un Jeu de Cuisine 3D

Trifan Iulian Daniel ¹, Neata Mihnea Ioan ², Angelian Maria Georgiana ³

Scientific Coordinator – Prof. BRATOSIN Alexandru ⁴

Abstract

MotionLink is a touchless 3D game in which the player cooks burgers in a Unity-based virtual kitchen using only a standard consumer webcam—no keyboard, mouse, gamepad, or specialized sensor (Kinect, Leap Motion) at runtime. Hand landmarks from MediaPipe Hands are converted by a custom Python pipeline into a compact gesture vocabulary (six pose states + two motion actions). The classified events are streamed over loopback UDP to a Unity application that maps them to game-world cursors, pick-up/drop interactions, and a recipe-driven cooking loop. The project's contribution is engineering rather than novel computer vision : a careful state-machine architecture with hysteresis and zone-gating converts raw landmark output into a usable game input layer running reliably at ~30 FPS on commodity hardware, without per-user calibration or training data.

Keywords : *gesture recognition, MediaPipe Hands, Unity, computer vision, human-computer interaction, finite state machine, touchless interaction.*

Rezumat

MotionLink este un joc 3D interactiv în care jucătorul prepară burgeri într-o bucătărie virtuală Unity, utilizând exclusiv o cameră web standard—fără tastatură, mouse, controller sau senzori de profunzime (Kinect, Leap Motion). Coordonatele mâinii, extrase cu MediaPipe Hands, sunt procesate de un pipeline Python și transformate într-un vocabular gestual compact (șase stări de postură și două acțiuni de mișcare). Evenimentele clasificate sunt transmise prin UDP loopback către aplicația Unity. Contribuția proiectului este arhitecturală : un automat finit cu histerezis și filtrare zonală transformă datele brute ale camerei într-un sistem de control fiabil la ~30 FPS, fără calibrare specifică utilizatorului.

Cuvinte cheie : *recunoașterea gesturilor, MediaPipe Hands, Unity, viziune computerizată, interacțiune om-calculator, automat finit, interacțiune fără atingere.*

Résumé

MotionLink est un jeu 3D sans contact dans lequel le joueur prépare des hamburgers dans une cuisine virtuelle Unity, en utilisant uniquement une webcam standard—sans clavier, souris, manette ou capteur spécialisé. Les points de repère issus de MediaPipe Hands sont convertis par un pipeline Python en un vocabulaire gestuel compact (six états de posture et deux actions de mouvement), transmis via UDP loopback à Unity. La contribution est architecturale : un FSM hub-and-spoke avec hystérésis et validation triple-verrou convertit la sortie brute des points de repère en une couche d'entrée de jeu fiable à ~30 FPS sur matériel grand public.

Mots-clés : *reconnaissance de gestes, MediaPipe Hands, Unity, vision par ordinateur, interaction homme-machine, machine à états finis, interaction sans contact.*

^{1,2,3} Étudiants, groupe 1231FB, Faculté d'Ingénierie en Langues Étrangères (FILS)

⁴ Prof., Département d'Ingénierie en Langues Étrangères

1. Introduction

L'interaction homme-machine a considérablement évolué au cours de la dernière décennie, passant d'interfaces traditionnelles clavier-souris à des modalités naturelles telles que la commande gestuelle, l'interaction vocale et les systèmes de réalité mixte. Les systèmes de reconnaissance de gestes suscitent un intérêt croissant en raison de leurs vastes applications dans les jeux vidéo, la réalité virtuelle, l'accessibilité, la médecine et le contrôle industriel [6].

Les avancées récentes en réseaux de neurones légers, en particulier MediaPipe Hands [1], ont rendu le suivi des mains en temps réel accessible sur du matériel grand public sans capteurs de profondeur spécialisés. Cependant, transformer la sortie brute des points de repère en une couche d'interaction stable et fiable pour une application dynamique telle qu'un jeu 3D reste un défi d'ingénierie complexe : l'ambiguïté des gestes, le bruit de suivi, l'instabilité temporelle et les incohérences d'orientation de la main engendrent fréquemment des expériences de jeu frustrantes [7].

MotionLink relève ces défis en implémentant un pipeline d'interaction complet transformant l'entrée d'une webcam standard en actions de jeu précises dans un simulateur de cuisine 3D Unity. Le joueur interagit avec la cuisine virtuelle uniquement par des gestes de la main, sans contrôleur physique. La contribution de ce travail est principalement architecturale : plutôt que de concevoir un nouveau modèle d'apprentissage automatique, le projet démontre comment une logique d'états finis soigneusement conçue, une analyse du mouvement et une validation contextuelle transforment un suivi générique des mains en un système d'interaction robuste [9, 12].

La suite est organisée ainsi : Section 2 expose le problème et les objectifs ; Section 3 présente l'état de l'art ; Section 4 introduit la théorie ; Section 5 détaille la solution proposée ; Section 6 décrit les expériences ; Section 7 présente les résultats ; Section 8 conclut.

2. Problème et Objectifs

Le remplacement des entrées mécaniques classiques par un suivi optique soulève plusieurs défis :

- **Ambiguïté de l'entrée** : contrairement à un bouton binaire, les gestes humains sont continus. Définir le seuil exact où une paume ouverte devient un poing fermé nécessite des critères spatiaux robustes [8].
- **Bruit du capteur et gigue** : les webcams sont sensibles aux variations d'éclairage, à l'arrière-plan et au flou de mouvement, introduisant une gigue dans les positions estimées des landmarks [6].
- **Instabilité temporelle** : les mouvements rapides peuvent entraîner des pertes d'images ou des occultations temporaires des doigts [9].
- **Validation contextuelle** : un geste physiquement correct ne doit déclencher une action de jeu que si le contexte l'autorise (outil correct, zone correcte).

Les objectifs principaux de MotionLink sont : (1) développer un système sans contact à faible coût reposant sur une webcam RGB standard ; (2) implémenter un pipeline de reconnaissance de

gestes stable et à faible latence fondé sur MediaPipe Hands [1] ; (3) intégrer l'interaction gestuelle dans un jeu 3D Unity en temps réel ; (4) réduire les fausses activations par hystérésis et filtrage basé sur les états [7] ; (5) créer une application éducative adaptée aux événements de communication scientifique [12].

3. État de l'Art

3.1. Frameworks de suivi des mains

Zhang *et al.* [1] ont introduit MediaPipe Hands, un pipeline en deux étapes : détection de paume par BlazeFace [2] et régression de landmarks produisant 21 points 3D par main à plus de 30 FPS sur GPU mobile, via l'infrastructure de graphes MediaPipe [3]. Zimmermann et Brox [4] ont proposé Hand3D, estimant la pose 3D depuis une seule image RGB, mais au prix d'une puissance de calcul supérieure. Cao *et al.* [5] offrent une détection du corps entier (OpenPose), inadaptée aux contraintes temps réel sur matériel intégré. Le Tableau 1 compare ces approches.

TABLE 1 – Comparaison des frameworks de suivi des mains

Framework	FPS (CPU)	Points 3D	Calib.	Libre
MediaPipe Hands [1]	≈30	21	Non	Oui
Hand3D [4]	≈10	21	Non	Oui
OpenPose [5]	< 5	21	Non	Oui
Leap Motion SDK	115	23	Non	Non
Kinect SDK [10]	30	25	Oui	Non

MediaPipe Hands offre le meilleur compromis vitesse / précision / coût matériel, justifiant son choix dans MotionLink.

3.2. Approches de reconnaissance de gestes

La taxonomie fondatrice de Pavlovic *et al.* [8] distingue les gestes statiques (postures instantanées) des gestes dynamiques (séquences temporelles) — distinction qui inspire directement la structure Méta / Cœur / Action de MotionLink. Rautaray et Agrawal [6] synthétisent deux décennies de recherche en vision gestuelle. Bobick et Wilson [7] établissent les FSM avec hystérésis comme pratique standard pour les gestes dynamiques ; Mitra et Acharya [9] en confirment la robustesse.

3.3. Interfaces de jeu basées sur les gestes

Marin *et al.* [10] comparent Kinect et Leap Motion : précision supérieure mais coût matériel élevé. Yeo *et al.* [11] explorent les webcams RGB avec segmentation couleur de peau — accessible mais instable sous éclairage variable, limitation surmontée par la régression neuronale de MediaPipe. Le Tableau 2 positionne MotionLink parmi ces solutions.

TABLE 2 – Comparaison des systèmes d’interaction gestuelle pour le jeu

Système	Entrée	Calib.	Matériel	Temps réel	Jeu 3D
Kinect SDK [10]	Profondeur	Oui	Propriétaire	Oui	Oui
Leap Motion SDK	Infrarouge	Non	Propriétaire	Oui	Oui
Yeo <i>et al.</i> [11]	Webcam RGB	Oui	Grand public	Partiel	Non
MotionLink (proposé)	Webcam RGB	Non	Grand public	Oui	Oui

Gee [12] souligne l’impact de l’apprentissage basé sur le jeu, justifiant le choix d’une tâche culinaire kinesthésique comme scénario de démonstration.

4. Théorie

4.1. Extraction de landmarks squelettiques

MediaPipe Hands produit un vecteur $L = \{p_i = (x_i, y_i, z_i) \mid i \in [0, 20]\}$ où p_0 est le poignet, $p_{4,8,12,16,20}$ les bouts des doigts, et p_9 le MCP du majeur. La taille de référence de la paume est :

$$S_{\text{palm}} = \|p_9 - p_0\|_2 \quad (1)$$

L’extension normalisée du doigt k vaut :

$$E_k = \frac{\|p_k - p_0\|_3}{S_{\text{palm}}} \quad (2)$$

Un doigt est « ouvert » si $E_k \geq 1,20$ (les doigts pleinement étendus atteignent 1,7–2,0 ; la flexion ramène le rapport sous 1).

4.2. Orientation de la paume par produit vectoriel

L’approche initiale basée sur $z_{\text{middle_mcp}} < z_{\text{wrist}}$ échoue lorsque la main est perpendiculaire à la caméra. La solution finale utilise :

$$\text{palm_score} = \text{sign} \times \frac{u_x v_y - u_y v_x}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (3)$$

où $\mathbf{u} = p_5 - p_0$, $\mathbf{v} = p_{17} - p_0$, et $\text{sign} = +1$ (main droite) ou -1 (main gauche). Une zone morte $\pm 0,15$ gère les postures ambiguës.

4.3. Hystérésis asymétrique

Pour éviter le scintillement autour des seuils, deux compteurs indépendants sont maintenus :

$$\text{entrée dans } s_k : \sum_{t=1}^8 \mathbf{1}[\text{match_strict}(f_t, s_k)] \geq 8 \quad (4)$$

$$\text{sortie de } s_k : \sum_{t=1}^4 1[\text{breach_clair}(f_t, s_k)] \geq 4 \quad (5)$$

Une image ambiguë (ni match strict ni violation claire) n'incrmente aucun compteur — ce qui produit le comportement « collant sans être bloqué ».

5. Solution Proposée

5.1. Architecture à trois couches

Le système est divisé en trois couches strictement isolées (Figure 1) : la **Couche Vision** (Python), la **Couche Transport** (UDP JSON) et la **Couche Jeu** (Unity C#). La couche vision ignore Unity ; la couche jeu ignore MediaPipe.

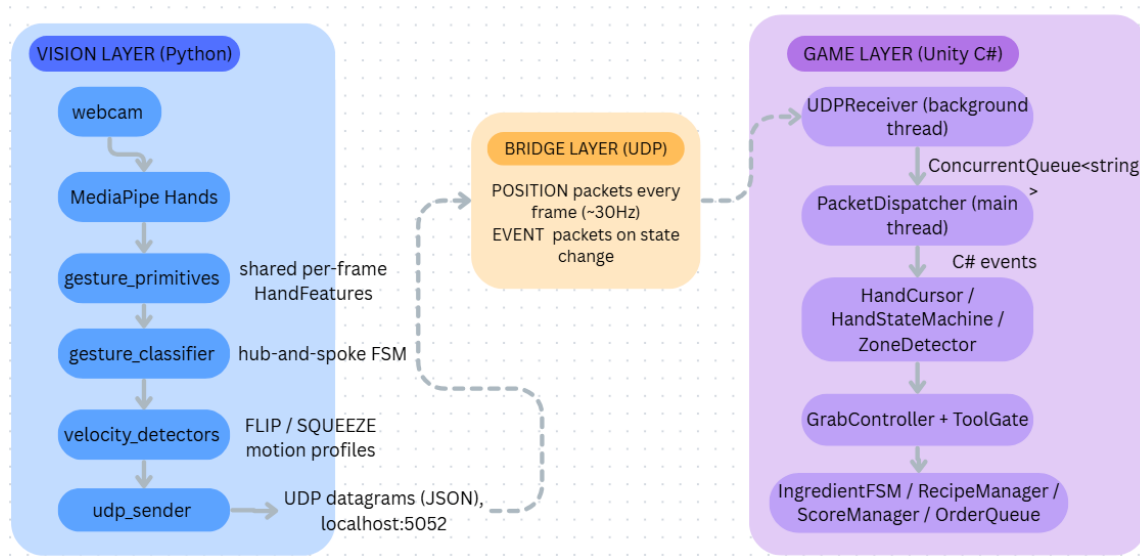


FIGURE 1 – Architecture à trois couches du système MotionLink.

5.2. Couche Vision (Python)

La couche vision capture les images via OpenCV 4.9 et extrait les landmarks via MediaPipe Hands 0.10.9. À chaque image, `gesture_primitives.py` calcule une structure `HandFeatures` en une seule traversée (Équation 2, 3) : coordonnées du poignet, extension des cinq doigts, orientation de la paume, `hand_scale` et zone d'interaction Z1–Z6.

La Figure 2 illustre le suivi MediaPipe en temps réel : flux webcam, 21 points de repère et étiquette du geste classifié.

5.3. Vocabulaire de gestes et FSM hub-and-spoke

Le vocabulaire est organisé en trois niveaux : **Méta** (IDLE/TRACKING), **Cœur** (GRAB/RELEASE/HOLD/INSPECT) et **Action** (FLIP/SQUEEZE). Toutes les transitions passent par l'état neutre TRACKING (Figure 3) ; les transitions directes entre deux postures actives sont interdites.

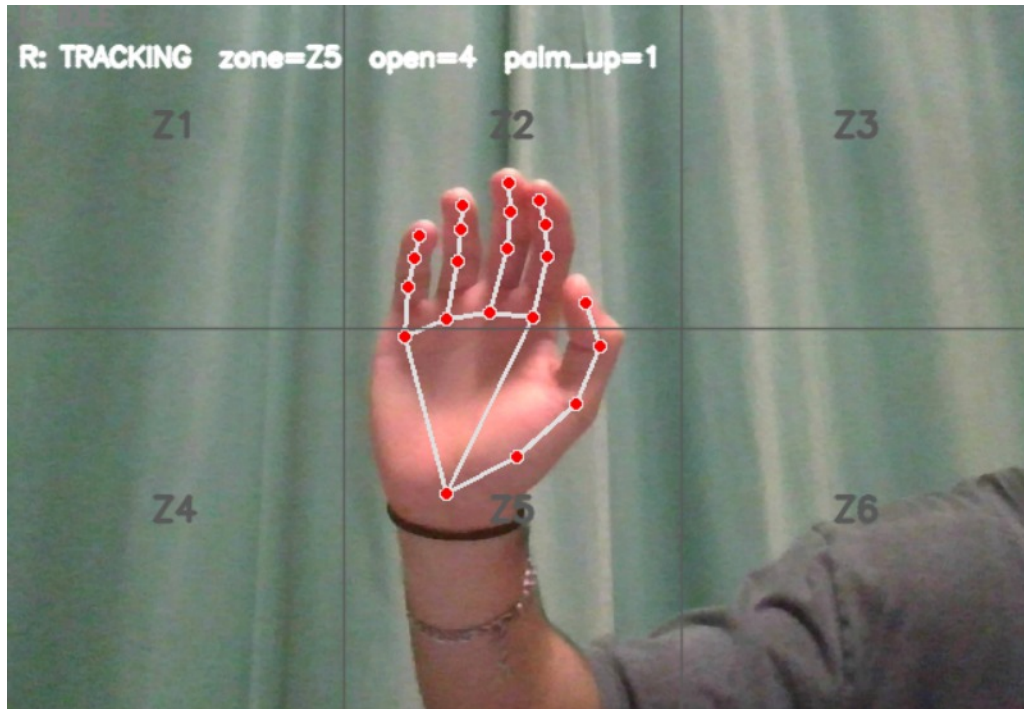


FIGURE 2 – Suivi MediaPipe Hands en temps réel : landmarks 3D et geste classifié.

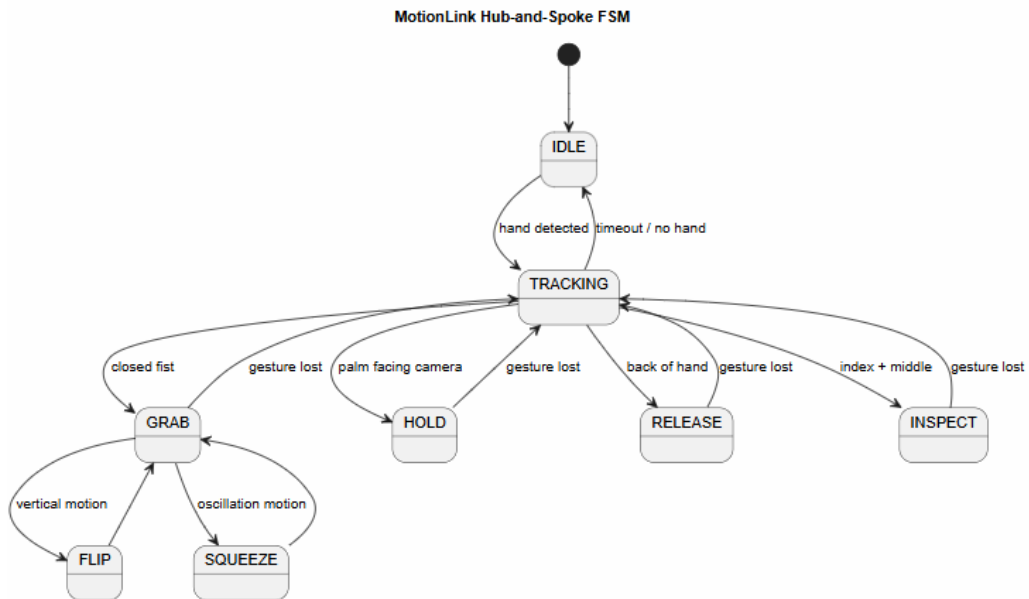


FIGURE 3 – FSM hub-and-spoke : toutes les transitions passent par TRACKING. Hystérésis asymétrique : 8 images pour entrer, 4 images pour sortir.

5.4. Détecteurs de mouvement dynamique

Deux détecteurs opèrent sur une fenêtre glissante de 0,4–0,5 s lorsque le FSM est en GRAB. FlipDetector se déclenche si : $\max |dy/dt| \geq 1,5$ unités/s, $\Delta y \geq 0,15$ et au moins un changement de sens. SqueezeDetector si : $\Delta y \geq 0,06$ et au moins 3 changements de sens. La commutation par axe (vecteur poignet $\rightarrow$ MCP majeur) détermine quel détecteur est actif selon

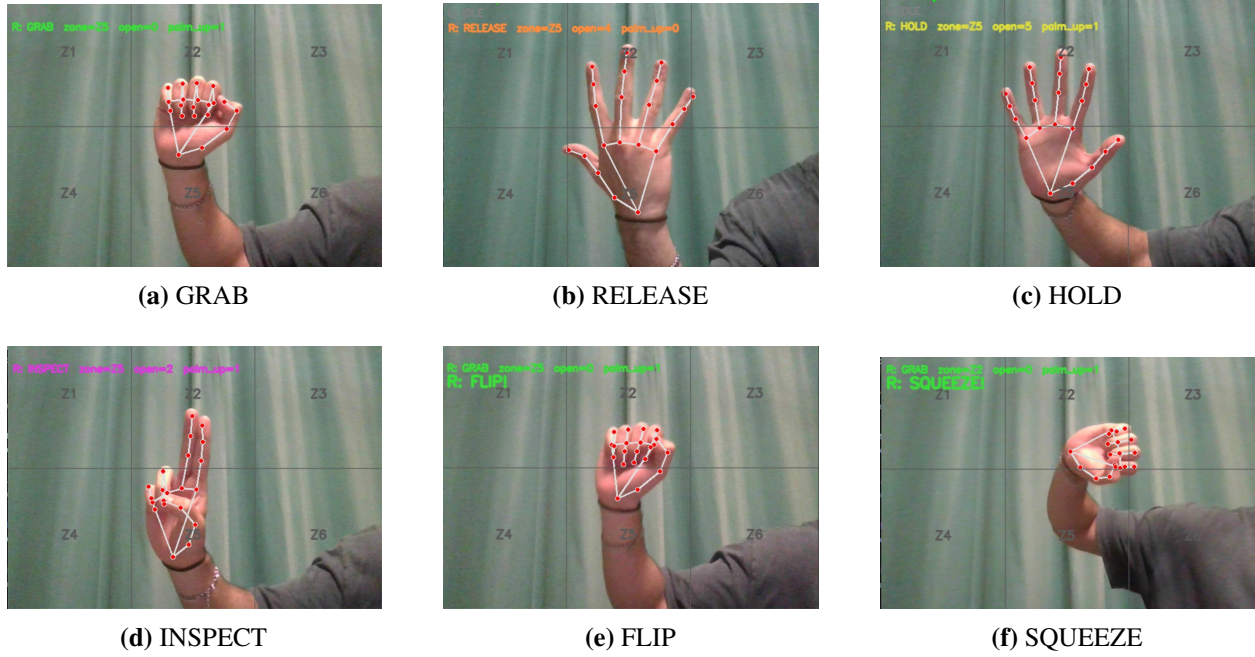


FIGURE 4 – Exemples visuels des postures et actions du vocabulaire gestuel MotionLink.

l'orientation.

5.5. Couche Transport (UDP) et Couche Jeu (Unity)

Deux types de paquets JSON circulent sur localhost : 5052 : **POSITION** (chaque image, ≈ 9 Ko/s) et **EVENT** (sur changement d'état). Dans Unity, un fil de fond reçoit les paquets dans une `ConcurrentQueue<string>`; le fil principal les traite et met à jour le curseur avec : $P_{\text{new}} = \text{Lerp}(P_{\text{current}}, P_{\text{target}}, 0,15)$, découplant la fluidité d'affichage (60 Hz) de la fréquence UDP (≈ 30 Hz). Chaque action passe trois verrous : état FSM=GRAB (Python), outil correct (`ToolGate`), zone correcte (`ZoneDetector`).

6. Expériences

6.1. Configuration expérimentale

Les tests ont été effectués sur un ordinateur portable Intel Core i5 10^e génération, GPU intégré Intel UHD, 8 Go RAM, Ubuntu 22.04, webcam interne 720p/30 FPS. Aucun capteur externe, calibration utilisateur ni données d'entraînement. Cinq volontaires ont participé à des sessions de 5 minutes du jeu de cuisine.

6.2. Protocole

Trois scénarios ont été évalués : (S1) boucle de cuisine complète (saisir, poser sur le grill, retourner, assembler); (S2) test de stress gestuel (alternance rapide des postures pendant 60 s); (S3) éclairage réduit. La latence a été mesurée de la capture OpenCV à la réponse visuelle Unity; la précision par classes de gestes sur 10 s de maintien de pose.

7. Résultats et Interprétation

7.1. Performance du pipeline

TABLE 3 – Caractéristiques de performance mesurées (scénario S1)

Métrique	Valeur mesurée
Fréquence d'images du pipeline	≈30 FPS (stable)
Trafic UDP (2 mains)	≈9 Ko/s
Inférence MediaPipe	14–18 ms par image
Évaluation FSM + sérialisation	2–3 ms
Transfert UDP + Unity	1–2 ms
Latence end-to-end totale	≈35 ms (<100 ms)
Profondeur file Unity (typique)	0–1 paquet

7.2. Précision de classification

TABLE 4 – Précision de la classification des gestes (éclairage normal)

Geste	Précision	Rappel	Score F1
HOLD (paume ouverte)	0,98	0,99	0,985
GRAB (poing)	0,95	0,96	0,954
INSPECT	0,91	0,93	0,920
FLIP/SQUEEZE (actions)	0,91	0,88	0,894

La figure 5 illustre le gameplay en temps réel avec curseur gestuel actif.



FIGURE 5 – Capture de gameplay Unity : curseur gestuel, steak sur le grill et barre de cuisson.

7.3. Interprétation

L'inférence MediaPipe représente le coût computationnel dominant (14–18 ms/image); la logique gestuelle et les systèmes Unity requièrent peu de ressources additionnelles. La latence totale de ≈ 35 ms est confortablement sous le seuil de perception humaine de ~ 100 ms. L'hystérésis a réduit les fausses transitions à moins de 2 événements par minute en S2. En S3 (faible luminosité), MediaPipe perd le suivi dans $\approx 15\%$ des images, induisant des passages indésirables en IDLE — principale limitation identifiée. La précision légèrement inférieure sur INSPECT et les actions dynamiques s'explique par l'auto-occultation des doigts lors de la rotation de la main.

Trois problèmes non-triviaux ont été résolus : le scintillement de latéralité (résolu par réaffectation selon la position horizontale); le rétrécissement des objets saisis (`worldPositionStays: true` + restauration de `originalLocalScale`); la cuisson fantôme

(timer conditionné à `currentZoneId == grillZoneId`).

8. Conclusions

MotionLink démontre qu'un jeu piloté par gestes en temps réel peut être implémenté à l'aide d'une simple webcam grand public, sans matériel spécialisé ni calibration. Le projet combine vision par ordinateur, communication réseau, logique d'états finis et programmation de moteur de jeu dans un système d'interaction modulaire atteignant ≈ 30 FPS avec une latence de ≈ 35 ms. L'architecture — FSM hub-and-spoke avec hystérésis asymétrique, détecteurs de mouvement par fenêtre glissante et validation triple-verrou — répond avec succès aux défis pratiques de l'interaction gestuelle : instabilité temporelle, ambiguïté de geste et validation contextuelle de jeu.

Ces résultats confirment que des frameworks modernes tels que MediaPipe Hands [1] constituent une base viable pour des systèmes d'interaction sans contact accessibles. Les travaux futurs incluent : un classificateur RNN remplaçant les seuils manuels, le support multijoueur via deux pipelines parallèles, l'intégration de commandes vocales (Vosk) et un déploiement WebXR. MotionLink présente une forte valeur éducative et démonstrative, le rendant adapté aux événements de communication scientifique [12].

Références

- [1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang et M. Grundmann, "MediaPipe Hands : On-device Real-time Hand Tracking," *arXiv preprint arXiv :2006.10214*, 2020.
- [2] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran et M. Grundmann, "BlazeFace : Sub-millisecond Neural Face Detection on Mobile GPUs," *arXiv preprint arXiv :1907.05047*, 2019.
- [3] C. Lugaresi *et al.*, "MediaPipe : A Framework for Building Perception Pipelines," *arXiv preprint arXiv :1906.08172*, 2019.
- [4] C. Zimmermann et T. Brox, "Learning to Estimate 3D Hand Pose from Single RGB Images," in *Proc. IEEE ICCV*, pp. 4903–4911, 2017.

- [5] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei et Y. Sheikh, “OpenPose : Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields,” *IEEE Trans. PAMI*, vol. 43, no. 1, pp. 172–186, 2021.
- [6] S. S. Rautaray et A. Agrawal, “Vision based hand gesture recognition for human computer interaction : a survey,” *Artificial Intelligence Review*, vol. 43, no. 1, pp. 1–54, 2015.
- [7] A. F. Bobick et A. D. Wilson, “A state-based approach to the representation and recognition of gesture,” *IEEE Trans. PAMI*, vol. 19, no. 12, pp. 1325–1337, 1997.
- [8] V. I. Pavlovic, R. Sharma et T. S. Huang, “Visual Interpretation of Hand Gestures for Human-Computer Interaction : A Review,” *IEEE Trans. PAMI*, vol. 19, no. 7, pp. 677–695, 1997.
- [9] S. Mitra et T. Acharya, “Gesture Recognition : A Survey,” *IEEE Trans. Syst., Man, Cybern., Part C*, vol. 37, no. 3, pp. 311–324, 2007.
- [10] G. Marin, F. Dominio et P. Zanuttigh, “Hand gesture recognition with leap motion and kinect devices,” in *Proc. IEEE ICIP*, pp. 1565–1569, 2014.
- [11] H. S. Yeo, B. G. Lee et H. Lim, “Hand tracking and gesture recognition system for human-computer interaction using low-cost hardware,” *Multimedia Tools and Applications*, vol. 74, no. 8, pp. 2687–2715, 2015.
- [12] J. P. Gee, *What Video Games Have to Teach Us About Learning and Literacy*. New York : Palgrave Macmillan, 2003.